

# Le basi

---

## Commenti

*Come scrivere note nel codice Java senza modificarne il comportamento.*

## A cosa servono i commenti

Un commento è una nota scritta nel codice. Java la ignora quando esegue il programma.

I commenti servono a spiegare un passaggio, lasciare un promemoria o rendere più chiara un'intenzione.

```
// Salviamo il nome dell'utente  
String nome = "Luca";
```

La riga che inizia con `//` non viene eseguita.

## Commenti su una riga

Il commento più semplice inizia con `//` e continua fino alla fine della riga.

```
public class Esempio {  
    public static void main(String[] args) {  
        // Questo messaggio viene mostrato nel terminale  
        System.out.println("Ciao!");  
    }  
}
```

Puoi usarlo anche alla fine di una riga:

```
int eta = 20; // eta dell'utente
```

Usalo con misura. Se il nome della variabile è già chiaro, il commento può essere inutile.

## Commenti su più righe

Un commento su più righe inizia con `/*` e finisce con `*/`.

```
/*
Questo programma calcola il totale
di un piccolo ordine.
*/
public class Totale {
    public static void main(String[] args) {
        double prezzo = 10.0;
        double spedizione = 3.5;
        System.out.println(prezzo + spedizione);
    }
}
```

È utile quando la nota è più lunga di una frase.

## Commenti Javadoc

Java ha anche un tipo speciale di commento: il commento Javadoc.

Inizia con `/**` e finisce con `*/`.

```
/**
 * Calcola il totale aggiungendo prezzo e spedizione.
 */
public static double calcolaTotale(double prezzo, double spedizione) {
    return prezzo + spedizione;
}
```

Questi commenti possono essere usati da strumenti automatici per generare documentazione.

All'inizio non devi usarli sempre. Ti basta sapere che esistono e che di solito descrivono classi e metodi.

## Commenti buoni e commenti inutili

Un commento è utile quando spiega il perché di una scelta.

```
// Applichiamo lo sconto solo se il totale supera 50 euro
if (totale > 50) {
    totale = totale - 5;
}
```

Questo commento aiuta.

Questo invece aggiunge poco:

```
// Aumenta eta di 1
eta = eta + 1;
```

Il codice è già abbastanza chiaro.

## Disattivare codice per provare

Durante gli esercizi puoi commentare una riga per non eseguirla temporaneamente:

```
System.out.println("Prima riga");
// System.out.println("Seconda riga");
System.out.println("Terza riga");
```

Output:

```
Prima riga
Terza riga
```

**Suggerimento:** va bene farlo mentre studi. Prima di tenere un programma ordinato, però, rimuovi il codice commentato che non serve più.

---

# Tipi di dato

*I principali tipi di dato in Java e quando usare numeri, testi e valori booleani.*

## Perche esistono i tipi

Ogni variabile contiene un tipo di informazione. Un prezzo non e uguale a un nome. Una risposta vero/falso non e uguale a una frase.

In Java devi dichiarare il **tipo di dato** della variabile:

```
int eta = 20;  
String nome = "Luca";  
boolean attivo = true;
```

Il tipo dice a Java che cosa puo contenere quella variabile e quali operazioni hanno senso.

## Numeri interi: int

`int` contiene numeri interi, senza virgola.

```
int eta = 20;  
int quantita = 3;  
int temperatura = -2;
```

Usalo per contatori, eta, quantita e punteggi.

```
int mele = 4;  
int pere = 2;  
int totale = mele + pere;  
  
System.out.println(totale);
```

Output:

6

## Numeri decimali: double

**double** contiene numeri con la parte decimale.

```
double prezzo = 12.50;  
double altezza = 1.75;
```

In Java i decimali usano il punto, non la virgola.

```
double prezzo = 10.0;  
double spedizione = 4.90;  
double totale = prezzo + spedizione;  
  
System.out.println(totale);
```

Output:

14.9

## Vero o falso: boolean

**boolean** contiene solo due valori:

```
true  
false
```

E' utile per rappresentare condizioni.

```
boolean maggiorenne = true;
boolean haSconto = false;

System.out.println(maggiorenne);
System.out.println(haSconto);
```

Output:

```
true
false
```

## Un singolo carattere: char

`char` contiene un solo carattere e usa gli apici singoli.

```
char iniziale = 'L';
char voto = 'A';
```

Non confonderlo con `String`, che usa le virgolette doppie e contiene testo anche lungo.

```
char lettera = 'A';
String parola = "A";
```

Sembrano simili, ma sono tipi diversi.

## Testo: String

`String` contiene testo.

```
String nome = "Luca";  
String messaggio = "Benvenuto";
```

Puoi unire testi con `+` :

```
String nome = "Luca";  
System.out.println("Ciao, " + nome);
```

Output:

```
Ciao, Luca
```

## Tipi primitivi e oggetti

Java ha tipi **primitivi**, come:

- `int`
- `double`
- `boolean`
- `char`

Sono tipi semplici, pensati per valori base.

`String` , invece, è un **oggetto**. Per ora ti basta sapere che `String` offre anche metodi utili:

```
String nome = "Luca";  
System.out.println(nome.length());
```

Output:

```
4
```

`length()` conta i caratteri del testo.

## Esempio completo

```
public class Tipi {  
    public static void main(String[] args) {  
        String prodotto = "Quaderno";  
        int quantita = 3;  
        double prezzo = 2.50;  
        boolean disponibile = true;  
  
        double totale = quantita * prezzo;  
  
        System.out.println("Prodotto: " + prodotto);  
        System.out.println("Quantita: " + quantita);  
        System.out.println("Totale: " + totale);  
        System.out.println("Disponibile: " + disponibile);  
    }  
}
```

Output:

```
Prodotto: Quaderno  
Quantita: 3  
Totale: 7.5  
Disponibile: true
```

Scegliere il tipo giusto rende il programma piu chiaro e aiuta Java a controllare gli errori.

---

## Primo programma

*Come scrivere, compilare ed eseguire un primo programma Java.*



## Il programma piu piccolo

Il primo programma serve a controllare che tutto funzioni e a vedere il percorso completo: scrivere, compilare, eseguire.

Crea un file chiamato `Ciao.java` :

```
public class Ciao {  
    public static void main(String[] args) {  
        System.out.println("Ciao!");  
    }  
}
```

Il nome del file deve essere `Ciao.java` perche la classe si chiama `Ciao` .

**Attenzione:** in Java maiuscole e minuscole contano. `Ciao` e `ciao` sono nomi diversi.

## Compilare il programma

Nel terminale, entra nella cartella dove si trova `Ciao.java` e scrivi:

```
javac Ciao.java
```

Se non vedi messaggi, e una buona notizia: significa che la compilazione e riuscita.

Java crea un nuovo file:

```
Ciao.class
```

Questo file non lo devi modificare a mano. E il risultato della compilazione.

## Eseguire il programma

Ora puoi eseguire il programma:

```
java Ciao
```

Output:

```
Ciao!
```

Quando usi `java Ciao`, stai dicendo alla JVM: "esegui la classe `Ciao`".

## Capire le righe principali

Vediamo il codice con calma.

```
public class Ciao {
```

Questa riga dichiara una **classe** chiamata `Ciao`. Per ora puoi pensare alla classe come al contenitore principale del programma.

```
public static void main(String[] args) {
```

Questa è la riga del metodo `main`. Java parte da qui quando esegui il programma.

```
System.out.println("Ciao!");
```

Questa riga stampa un testo nel terminale.

Le parentesi graffe `{` e `}` indicano dove inizia e finisce un blocco di codice.

## Modificare il messaggio

Prova a cambiare il testo:

```
public class Ciao {  
    public static void main(String[] args) {  
        System.out.println("Sto imparando Java.");  
    }  
}
```

Ricompila:

```
javac Ciao.java
```

Poi esegui:

```
java Ciao
```

Output:

```
Sto imparando Java.
```

Ogni volta che modifichi il file `.java`, devi ricompilare prima di eseguire la nuova versione.

## Errori comuni

Se dimentichi il punto e virgola:

```
System.out.println("Ciao!")
```

Java segnala un errore di compilazione. La versione corretta è:

```
System.out.println("Ciao!");
```

Se il nome del file non corrisponde alla classe pubblica, Java segnala un altro errore. Con `public class Ciao`, il file deve chiamarsi `Ciao.java`.

---

## Installazione

*Come installare Java, configurare il JDK e preparare l'ambiente di lavoro.*

## Cosa devi installare

Per programmare in Java ti serve il **JDK**, cioè il Java Development Kit.

Il JDK contiene gli strumenti per:

- compilare codice Java
- eseguire programmi Java
- usare le librerie standard del linguaggio

Potresti sentire anche la parola **JRE**, Java Runtime Environment. Il JRE serve solo a eseguire programmi Java già pronti. Per scrivere codice, invece, ti serve il JDK.

**Nota:** se stai imparando Java, scegli sempre il JDK. E il pacchetto completo.

## Installare il JDK

Puoi usare un JDK moderno, per esempio una versione LTS, cioè con supporto a lungo termine.

Le distribuzioni più comuni sono:

- Eclipse Temurin
- Oracle JDK
- Microsoft Build of OpenJDK
- Amazon Corretto

Per iniziare, una qualunque di queste va bene. L'importante è installare un JDK, non solo un JRE.

## Controllare l'installazione

Dopo l'installazione, apri il terminale.

Su Windows puoi usare Prompt dei comandi o PowerShell. Su macOS e Linux puoi usare Terminale.

Scrivi:

```
java --version
```

Se Java è installato, vedrai un numero di versione.

Poi controlla anche il compilatore:

```
javac --version
```

`java` esegue i programmi. `javac` li compila.

Se `java` funziona ma `javac` no, probabilmente hai installato un runtime e non un JDK, oppure il percorso degli strumenti non è configurato correttamente.

## Scegliere un editor

Puoi scrivere Java con molti editor. Per iniziare, Visual Studio Code va bene perché è leggero e gratuito.

Installa:

1. Visual Studio Code
2. L'estensione "Extension Pack for Java"

Puoi usare anche IntelliJ IDEA Community Edition. È molto usato per Java e offre molti aiuti automatici, ma all'inizio può sembrare più ricco del necessario.

## Creare una cartella di lavoro

Crea una cartella per gli esercizi, per esempio:

```
java-esercizi
```

Dentro questa cartella metterai i file `.java`.

Un file Java contiene codice sorgente, cioè il testo che scrivi tu. Quando lo compili, Java crea un file `.class`, che contiene il codice pronto per la JVM.

## Primo controllo completo

Crea un file chiamato `Saluto.java` con questo contenuto:

```
public class Saluto {  
    public static void main(String[] args) {  
        System.out.println("Java funziona!");  
    }  
}
```

Nel terminale, entra nella cartella dove hai salvato il file e compila:

```
javac Saluto.java
```

Poi esegui:

```
java Saluto
```

Output:

```
Java funziona!
```

**Attenzione:** quando esegui il programma scrivi `java Saluto`, senza `.java` e senza `.class`.

Se vedi il messaggio, l'ambiente è pronto.

# Operatori

*Come combinare valori con operatori aritmetici, di confronto e logici.*

## Cosa sono gli operatori

Gli operatori sono simboli che fanno qualcosa con i valori.

Hai già visto `=` :

```
int eta = 20;
```

Qui `=` assegna il valore `20` alla variabile `eta` .

Java ha operatori per calcolare, confrontare e combinare condizioni.

## Operatori aritmetici

Gli operatori aritmetici lavorano con i numeri.

```
int a = 10;
int b = 3;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Output:

```
13
7
30
3
1
```

Il simbolo `%` si chiama **modulo** e restituisce il resto della divisione.

`10 % 3` vale `1`, perché 10 diviso 3 fa 3 con resto 1.

## Divisione tra interi

Con due `int`, Java fa una divisione intera.

```
int risultato = 10 / 3;  
System.out.println(risultato);
```

Output:

```
3
```

La parte decimale viene tagliata.

Se vuoi un risultato decimale, usa almeno un `double`:

```
double risultato = 10.0 / 3;  
System.out.println(risultato);
```

## Assegnazione

`=` assegna un valore.

```
int punti = 10;  
punti = punti + 5;
```

Java offre anche forme più brevi:



```
punti += 5; // come punti = punti + 5
punti -= 2; // come punti = punti - 2
```

Per aumentare o diminuire di 1:

```
punti++;
punti--;
```

All'inizio usa pure la forma lunga. E più esplicita.

## Operatori di confronto

Gli operatori di confronto producono un valore `boolean`: `true` oppure `false`.

```
int eta = 20;

System.out.println(eta > 18);
System.out.println(eta == 18);
System.out.println(eta != 18);
```

Output:

```
true
false
true
```

I principali sono:

- `>` maggiore di
- `<` minore di
- `>=` maggiore o uguale
- `<=` minore o uguale
- `==` uguale

- `!=` diverso

**Attenzione:** `=` assegna. `==` confronta.

## Operatori logici

Gli operatori logici combinano condizioni.

```
int eta = 20;
boolean haBiglietto = true;

boolean puoEntrare = eta >= 18 && haBiglietto;
System.out.println(puoEntrare);
```

Output:

```
true
```

I principali operatori logici sono:

- `&&` significa "e": entrambe le condizioni devono essere vere
- `||` significa "oppure": basta una condizione vera
- `!` significa "non": inverte il valore

```
boolean piove = false;
System.out.println(!piove);
```

Output:

```
true
```

# Precedenza

Java esegue alcune operazioni prima di altre.

```
int risultato = 2 + 3 * 4;  
System.out.println(risultato);
```

Output:

14

La moltiplicazione avviene prima dell'addizione.

Se vuoi rendere l'ordine chiaro, usa le parentesi:

```
int risultato = (2 + 3) * 4;  
System.out.println(risultato);
```

Output:

20

Le parentesi non servono solo al computer. Servono anche a chi legge il codice.

---

## Struttura di un programma

*Come sono organizzati classi, metodi e istruzioni in un file Java.*

## Guardare un programma a strati

Un programma Java sembra lungo anche quando fa poco. Questo succede perché Java vuole una struttura esplicita.

Partiamo da un esempio:

```
public class Programma {  
    public static void main(String[] args) {  
        System.out.println("Inizio");  
        System.out.println("Fine");  
    }  
}
```

Puoi leggerlo a strati:

- la classe contiene il programma
- il metodo `main` contiene le istruzioni da eseguire
- le istruzioni sono le singole azioni

## La classe

```
public class Programma {  
    ...  
}
```

Una **classe** è un blocco che raccoglie codice. Nei primi esempi useremo una classe come contenitore principale.

Il nome della classe pubblica deve corrispondere al nome del file.

Se scrivi:

```
public class Programma {  
}
```

il file deve chiamarsi:

```
Programma.java
```

## Il metodo main

```
public static void main(String[] args) {  
    ...  
}
```

Il metodo `main` è il punto di partenza del programma. Quando scrivi `java Programma`, Java cerca questo metodo e inizia da lì.

Per ora non serve capire ogni parola della riga. Tienila come formula di avvio:

```
public static void main(String[] args)
```

La capirai meglio quando parleremo di metodi, classi e oggetti.

## Le istruzioni

Dentro `main` scriviamo le istruzioni:

```
System.out.println("Inizio");  
System.out.println("Fine");
```

Una **istruzione** è un comando che Java deve eseguire.

In Java molte istruzioni finiscono con il punto e virgola `;`.

```
int eta = 20;  
System.out.println(eta);
```

Il punto e virgola dice a Java: "questa istruzione è finita".

# Le parentesi graffe

Le parentesi graffe delimitano un blocco:

```
{  
    // codice dentro il blocco  
}
```

Nel nostro programma ci sono due blocchi:

```
public class Programma {  
    public static void main(String[] args) {  
        System.out.println("Ciao");  
    }  
}
```

Il blocco di `main` sta dentro il blocco della classe.

L'indentazione, cioè gli spazi all'inizio delle righe, non cambia il significato per Java. Serve però a noi per leggere meglio.

## Un programma con più istruzioni

```
public class Scontrino {  
    public static void main(String[] args) {  
        double prezzo = 12.50;  
        double spedizione = 4.90;  
        double totale = prezzo + spedizione;  
  
        System.out.println("Prezzo: " + prezzo);  
        System.out.println("Spedizione: " + spedizione);  
        System.out.println("Totale: " + totale);  
    }  
}
```

Output:

```
Prezzo: 12.5  
Spedizione: 4.9  
Totale: 17.4
```

Il programma viene eseguito dall'alto verso il basso, una riga alla volta.

## Regola pratica

Quando scrivi un file Java di base, controlla tre cose:

1. il nome del file corrisponde al nome della classe
2. il metodo `main` è scritto correttamente
3. ogni istruzione che lo richiede finisce con `;`

Molti errori iniziali nascono da uno di questi tre dettagli.

## Stringhe

*Come lavorare con testi, concatenazione e metodi comuni di String.*

## Cosa è una stringa

Una stringa è un testo. In Java il tipo per i testi è `String`.

```
String nome = "Luca";  
String messaggio = "Benvenuto";
```

Le stringhe usano le virgolette doppie.

```
String parola = "ciao";
```

Gli apici singoli, invece, servono per un solo carattere:

```
char iniziale = 'L';
```

## Concatenare stringhe

Per unire stringhe, usa `+`.

```
String nome = "Luca";  
String saluto = "Ciao, " + nome + "!";  
  
System.out.println(saluto);
```

Output:

```
Ciao, Luca!
```

Puoi unire anche numeri:

```
int eta = 20;  
System.out.println("Eta: " + eta);
```

Java converte il numero in testo per costruire il messaggio.

## Contare i caratteri con length

Le stringhe hanno metodi, cioè azioni che puoi chiamare sul valore.

`length()` conta i caratteri:

```
String nome = "Sara";  
System.out.println(nome.length());
```



Output:

```
4
```

La chiamata si legge così: "sulla stringa `nome` , esegui il metodo `length` ".

## Confrontare stringhe con equals

Per confrontare il contenuto di due stringhe, usa `equals` .

```
String password = "segreto";  
  
System.out.println(password.equals("segreto"));  
System.out.println(password.equals("ciao"));
```

Output:

```
true  
false
```

**Attenzione:** con le stringhe evita `==` per confrontare il testo. Usa `equals` .

`==` controlla se due riferimenti puntano allo stesso oggetto. `equals` controlla se il contenuto è uguale.

Per iniziare, ricordati questa regola pratica:

```
testo.equals("valore")
```

## Prendere una parte con substring

`substring` prende una parte del testo.

```
String parola = "programmare";
String parte = parola.substring(0, 7);

System.out.println(parte);
```

Output:

```
program
```

Gli indici partono da `0`. Il primo numero è incluso, il secondo è escluso.

In `"programmare"` :

- indice `0` : p
- indice `1` : r
- indice `2` : o

`substring(0, 7)` prende i caratteri dagli indici 0 a 6.

## Cambiare maiuscole e minuscole

Altri metodi utili:

```
String nome = "Luca";

System.out.println(nome.toUpperCase());
System.out.println(nome.toLowerCase());
```

Output:

```
LUCA
luca
```

Questi metodi non modificano la stringa originale. Creano un nuovo testo.

## Esempio completo

```
public class NomeUtente {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Nome: ");  
        String nome = scanner.nextLine();  
  
        System.out.println("Ciao, " + nome + "!");  
        System.out.println("Il tuo nome ha " + nome.length() + "  
caratteri.");  
        System.out.println("In maiuscolo: " + nome.toUpperCase());  
  
        scanner.close();  
    }  
}
```

Le stringhe saranno ovunque nei programmi Java: messaggi, input, nomi, percorsi di file e dati letti dall'esterno.

## Conversione dei tipi

*Come convertire valori tra tipi diversi ed evitare errori comuni.*

## Perche convertire un tipo

A volte hai un valore in un tipo, ma ti serve in un altro.

Per esempio:

- leggi un numero come testo e vuoi usarlo per un calcolo
- hai un `int` e vuoi salvarlo in un `double`
- hai un `double` e vuoi prendere solo la parte intera

Questa operazione si chiama **conversione di tipo**.

# Conversioni automatiche

Java permette alcune conversioni senza istruzioni speciali.

Per esempio, un `int` può diventare un `double` :

```
int eta = 20;
double etaDecimale = eta;

System.out.println(etaDecimale);
```

Output:

```
20.0
```

Questa conversione è sicura: un numero intero può essere rappresentato come numero decimale senza perdere informazione.

## Cast esplicito

La conversione opposta non è automatica.

```
double prezzo = 12.90;
int prezzoIntero = prezzo; // errore
```

Java non vuole perdere la parte decimale senza che tu lo dica chiaramente.

Per farlo usi un **cast**:

```
double prezzo = 12.90;
int prezzoIntero = (int) prezzo;

System.out.println(prezzoIntero);
```

Output:

12

Il cast a `int` taglia la parte decimale. Non arrotonda.

## Perdita di informazioni

Questo dettaglio è importante:

```
double media = 7.8;
int voto = (int) media;

System.out.println(voto);
```

Output:

7

La parte `.8` sparisce.

Se vuoi arrotondare, usa un metodo apposito:

```
double media = 7.8;
long arrotondato = Math.round(media);

System.out.println(arrotondato);
```

Output:

8

# Convertire testo in numero

Quando leggi input dall'utente o da un file, spesso i dati arrivano come testo.

Per trasformare una `String` in un `int`, usa `Integer.parseInt`:

```
String testo = "25";  
int numero = Integer.parseInt(testo);  
  
System.out.println(numero + 5);
```

Output:

```
30
```

Per un `double`, usa `Double.parseDouble`:

```
String testo = "12.50";  
double prezzo = Double.parseDouble(testo);  
  
System.out.println(prezzo * 2);
```

Output:

```
25.0
```

## Errore comune: testo non valido

Se il testo non contiene un numero valido, Java segnala un errore.

```
String testo = "ciao";  
int numero = Integer.parseInt(testo); // errore a runtime
```

"ciao" non può diventare un numero intero.

Più avanti vedremo come gestire questi casi con le eccezioni.

## Convertire numero in testo

Per unire un numero a una stringa, spesso basta `+` :

```
int eta = 20;  
String messaggio = "Eta: " + eta;  
  
System.out.println(messaggio);
```

Output:

```
Eta: 20
```

Java converte il numero in testo per poter costruire la frase.

## Regola pratica

Chiediti sempre:

- sto passando da un tipo più piccolo a uno più ampio?
- rischio di perdere dati?
- il testo contiene davvero un numero?

Se la conversione può perdere informazione, Java ti chiede di essere esplicito. È una protezione, non un fastidio.

# Variabili

*Come salvare valori in Java, dichiarare variabili e modificarle nel tempo.*

## Cos'è una variabile

Un programma deve ricordare informazioni mentre lavora: un nome, un prezzo, un punteggio, un totale.

Una **variabile** è una specie di scatola con un'etichetta. L'etichetta è il nome della variabile. Il contenuto è il valore salvato.

```
String nome = "Luca";  
int eta = 20;
```

Qui creiamo due variabili:

- `nome` contiene il testo `"Luca"`
- `eta` contiene il numero `20`

## Dichiarare una variabile

In Java devi indicare il tipo della variabile prima del nome.

```
int eta;
```

Questa riga dice: "crea una variabile chiamata `eta` che conterrà numeri interi".

Puoi anche darle subito un valore:

```
int eta = 20;
```

Questa operazione si chiama **inizializzazione**.



# Leggere il valore

Per usare il valore, scrivi il nome della variabile.

```
public class Variabili {  
    public static void main(String[] args) {  
        String nome = "Luca";  
        int eta = 20;  
  
        System.out.println(nome);  
        System.out.println(eta);  
    }  
}
```

Output:

```
Luca  
20
```

Quando Java vede `nome`, prende il valore salvato dentro quella variabile.

# Modificare una variabile

Puoi cambiare il valore di una variabile con `=`.

```
int eta = 20;  
System.out.println(eta);  
  
eta = 21;  
System.out.println(eta);
```

Output:

```
20  
21
```

La seconda assegnazione sostituisce il valore precedente.

Puoi anche usare il valore attuale per calcolarne uno nuovo:

```
int punti = 10;  
punti = punti + 5;  
  
System.out.println(punti);
```

Output:

```
15
```

## Il tipo conta

Java controlla che il valore sia compatibile con il tipo.

```
int eta = 20;  
eta = "venti"; // errore
```

`int` contiene numeri interi. Non può contenere testo.

Questo controllo può sembrare rigido, ma aiuta a trovare molti errori prima di eseguire il programma.

## Nomi validi

I nomi delle variabili devono seguire alcune regole:

- possono contenere lettere, numeri e underscore `_`
- non possono iniziare con un numero
- non possono contenere spazi

- non possono essere parole riservate come `class`, `int`, `if`
- maiuscole e minuscole contano

Esempi validi:

```
String nome;  
int etaUtente;  
double prezzoTotale;
```

Esempi non validi:

```
int 2eta;           // inizia con un numero  
String nome utente; // contiene uno spazio  
int class;          // parola riservata
```

## Convenzione: camelCase

In Java i nomi delle variabili usano spesso il **camelCase**.

```
int etaMinima = 18;  
double prezzoTotale = 24.90;  
String nomeUtente = "Sara";
```

La prima parola inizia minuscola. Le parole successive iniziano con la maiuscola.

## Esempio completo

```
public class Profilo {  
    public static void main(String[] args) {  
        String nome = "Sara";  
        int eta = 17;  
        boolean studente = true;  
  
        System.out.println("Nome: " + nome);  
        System.out.println("Eta: " + eta);  
        System.out.println("Studente: " + studente);  
  
        eta = eta + 1;  
        System.out.println("Eta aggiornata: " + eta);  
    }  
}
```

Output:

```
Nome: Sara  
Eta: 17  
Studente: true  
Eta aggiornata: 18
```

Qui `eta` cambia, mentre `nome` e `studente` restano uguali.